

Desarrollo de un sistema de programación automatizada de tareas de mantenimiento en la industria minera

This is a subtitle

Emilio Bravo Maturana

¹ROA, Maastricht University

²SBE, Maastricht University

5 de enero de 2025



Maastricht University



Agenda

- ① Resumen Ejecutivo
- ② Análisis y descripción del problema
- ③ Definiciones teóricas
- ④ Aplicación de los conceptos teóricos al problema
- ⑤ Requisitos de la solución
- ⑥ Diseño de la solución
- ⑦ Resultados y análisis
- ⑧ Conclusiones y pasos a seguir

1. Resumen Ejecutivo

Programación de Actividades de Mantenimiento

- El mantenimiento eficiente de equipos es crucial para garantizar la continuidad operativa en cualquier industria.
- Las **restricciones de recursos** y las ventanas temporales son factores que complejizan el proceso de programación.
- La programación manual de actividades de mantenimiento:
 - Es compleja e intensiva en tiempo.
 - Es propensa a errores debido a múltiples variables.
 - Depende altamente de personal especializado.

Propuesta de Automatización

Objetivo Principal

Automatizar la programación de tareas de mantenimiento mediante herramientas avanzadas de modelado y optimización.

- El problema se modela como un Problema de Programación de Proyectos con Restricciones de Recursos (*RCPSP*).
- Se utilizan técnicas de Programación por Restricciones (*Constraint Programming, CP*) para encontrar soluciones eficientes.

Programación de Actividades

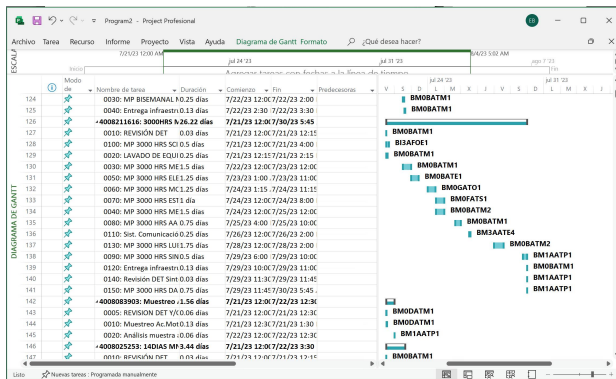


Figura: Captura de Cronograma Resultante en MS Project ?.

Impacto de la Solución

Un correcto diseño e implementación de la solución propuesta permitirá:

- **Optimización del tiempo y esfuerzo:** Liberación del equipo de programación de tareas manuales intensivas.
- **Reducción de errores:** Especialmente aquellos derivados de programaciones deficientes.

Resultados Obtenidos

- Cronograma generado para **4.898 tareas**.
- Programación del 98,6 % de las tareas.
- *Makespan* (Tiempo total del proyecto): 21,5 días.
- Tiempo de procesamiento: **6,12 segundos**.

2. Análisis y descripción del problema

Levantamiento del Proceso de Programación

- **Objetivo:** Generalizar el proceso de programación de tareas en la empresa colaboradora.
- **Metodología:**
 - Análisis de documentos normativos del proceso.
 - Encuesta dirigida al personal encargado de la programación.
- Este análisis permitió:
 - Comprender el funcionamiento operativo actual.
 - Identificar barreras y desafíos.
 - Validar la necesidad de una solución automatizada.

Ciclo Semanal de Programación

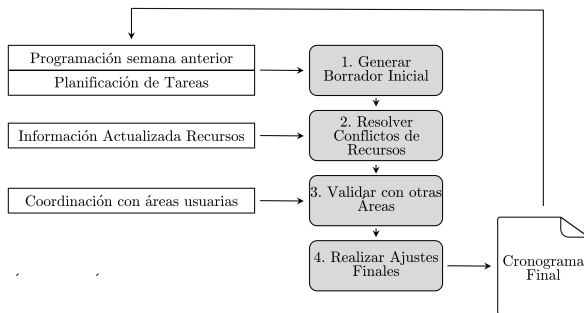


Figura: Ciclo Semanal de Programación de Actividades.

Proceso de Programación

- Recibe la planificación inicial como *input* para ajustar a las capacidades reales.
- Tareas principales:
 - **Generar el borrador inicial:** Fecha y hora de inicio para cada tarea.
 - **Resolver conflictos de recursos:** Evitar sobreasignaciones.
 - **Validar con otras áreas:** Asegurar viabilidad del cronograma.
 - **Realizar ajustes finales:** Incorporar cambios de último minuto.
- El cronograma resultante detalla tareas, fechas, duración y recursos, priorizando la continuidad operativa.
- Cada viernes se emite el cronograma oficial, base del siguiente ciclo iterativo.

Diagnóstico del Proceso Actual

Se llevó a cabo un diagnóstico del proceso de programación de tareas a través de una **encuesta** al personal de programación, identificando:

- 64 % invierte más de la mitad de su jornada en la generación de cronogramas.
- La fase más intensiva es la **creación del borrador inicial**.
- El error más recurrente es la **no disponibilidad de materiales y repuestos**.
- **Entorno dinámico**: 90 % afronta cambios de último minuto al menos una vez por semana.
- Existen barreras de **coordinación** y **colaboración** con otras áreas.

Distribución del Tiempo de Programación

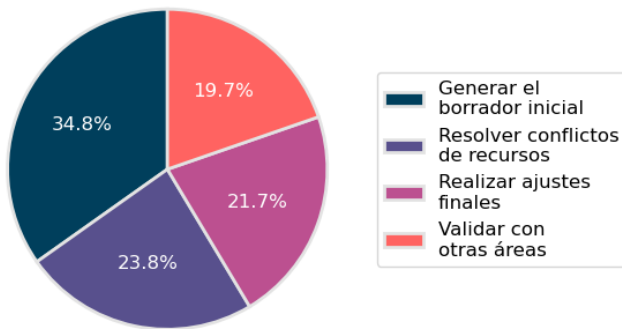


Figura: Distribución del tiempo de programación

Fuente: Elaboración propia en base a encuesta

3. Definiciones teóricas

Definiciones Teóricas

Se presentan los conceptos fundamentales para comprender el enfoque del proyecto.

- **RCPSP** (Problema de Programación de Proyectos con Restricciones de Recursos)
- **CP** (Programación por Restricciones)

RCPSP: Definición e Importancia

- El ***RCPSP*** es un problema clásico de investigación operativa y gestión de proyectos.
- Pertenece a los *Constraint Satisfaction Problems* (CSP) y, más específicamente, a los *Constraint Optimization Problems* (COP).

Ejemplo de *RCPS*

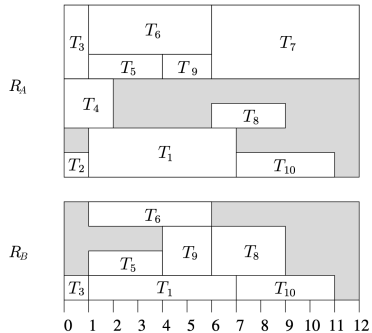


Figura: Ejemplo de un *RCPS* con 2 recursos y 10 tareas.

Características principales del *RCPSP*

- **Restricciones de precedencia:**

- Algunas actividades no pueden comenzar hasta que otras hayan finalizado.

- **Recursos limitados:**

- Mano de obra, equipos y materiales con disponibilidad limitada cada periodo.

- **Objetivo de optimización:**

- Generalmente, minimizar el tiempo total del proyecto (*makespan*).

- También pueden considerarse otros objetivos (costos, carga de recursos, etc.).

Complejidad del *RCPSP*

- El *RCPSP* es un problema *NP-Hard*, lo que implica un crecimiento exponencial del espacio de soluciones conforme aumenta el número de tareas y recursos.
- Para instancias pequeñas, se pueden aplicar métodos exactos; sin embargo, a mayor escala, se requieren enfoques alternativos.
- **Programación por Restricciones (CP)** se presenta como una herramienta eficaz para modelar y resolver el *RCPSP* de manera declarativa.

***CP*: Definición e Importancia**

- ***CP*** es una técnica para resolver problemas combinatorios complejos (planificación, programación, optimización).
- En este trabajo, *CP* es fundamental para abordar el *RCPSP*, permitiendo modelar restricciones de tiempo y recursos.

Esquema de Programación por Restricciones

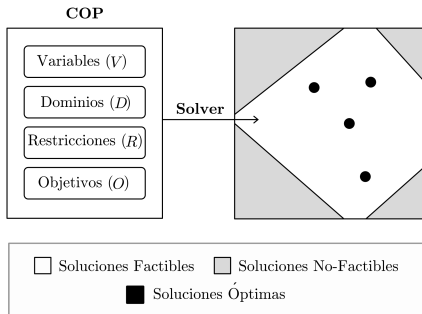


Figura: Esquema de Programación con Restricciones.

4. Aplicación de los conceptos teóricos al problema

Modelado como un *RCPSP*

- **Objetivo:** Asignar tiempos de inicio a cada tarea, minimizando el *makespan* y la cantidad de tareas no programadas.
- Restricciones de:
 - Capacidad de recursos.
 - Ventanas de tiempo y precedencia.
 - Intervalos prohibidos.

Parámetros del modelo

- **Tareas** $T = \{T_0, T_1, \dots, T_n\}$:
 - Duración $C(T_i)$.
 - Recursos requeridos $D(T_i)$ y cantidad q_i .
 - Criticidad Impact_i .
 - Ventana de tiempo $W(T_i) = [\text{early}_i, \text{late}_i]$.
- **Recursos** $R = \{R_0, R_1, \dots, R_m\}$:
 - Capacidad $P(R_j)$.
 - Intervalos prohibidos $I(R_j)$.
- **Grupos de tareas** $G = \{G_0, G_1, \dots\}$:
 - Conjunto de tareas con restricciones de precedencia internas.

Variables de decisión

- $x_i \in \{0, 1\}$: Indica si la tarea T_i es **programada**.
- $S_i \in \mathbb{N}$: Momento de **inicio** de la tarea T_i .

Restricciones (I)

1 Ventanas de tiempo:

$$x_i = 1 \implies \text{early}_i \leq S_i \leq \text{late}_i - C(T_i)$$

2 Capacidad de recursos:

$$\sum_{\substack{T_i \in T \\ R_j \in D(T_i)}} q_i \cdot \delta_{ij}(t) \leq P(R_j), \quad \forall t \in \text{Horizonte}$$

donde $\delta_{ij}(t) = 1$ si $t \in [S_i, E_i]$ y $x_i = 1$, en caso contrario $\delta_{ij}(t) = 0$.

Restricciones (II)

3 Intervalos prohibidos:

$$x_i = 1 \implies \forall R_j \in D(T_i), \forall [a_k, b_k) \in I(R_j) : [S_i, E_i) \cap [a_k, b_k) = \emptyset$$

4 Restricciones de precedencia en grupos:

$$y_k = 1 \implies \forall (T_i, T_{i+1}) \in G_k : S_{i+1} \geq E_i \quad \text{y} \quad \forall T_i \in G_k : x_i = y_k$$

Función objetivo

- Se desea minimizar *makespan* y penalizar la no programación de tareas.
- Definimos un **peso** para cada tarea basado en su criticidad:

$$w_i = (\text{Impact}_{\max} + 1 - \text{Impact}_i)^3$$

donde $\text{Impact}_{\max} = 3$.

- **Función objetivo:**

$$\text{mín } Z = \alpha \cdot \text{makespan} + \beta \cdot \sum_{i=0}^n w_i \cdot (1 - x_i)$$

→ α : Peso asociado al *makespan*.

→ β : Peso asociado a la no-programación de tareas (según criticidad).

Detalle de la función objetivo

$$\text{mín } Z = \alpha \cdot \text{makespan} + \beta \cdot \sum_{i=0}^n w_i \cdot (1 - x_i)$$

1 Componente Makespan ($\alpha \cdot \text{makespan}$):

- Minimiza la extensión global del horizonte temporal.
- Promueve la eficiencia en el uso de recursos.

2 Componente Penalización ($\beta \cdot \sum w_i(1 - x_i)$):

- Permite que ciertas tareas no se programen, pero **penaliza** fuertemente la no-programación de tareas críticas.
- Evita que el modelo sea infactible si hay incompatibilidades extremas.

5. Requisitos de la solución

Datos necesarios para la programación

- El *input* principal proviene del proceso de **planificación**, que define:
 - Ventana de fechas.
 - Duración y criticidad de cada tarea.
 - Recursos requeridos (cuadrillas, equipo auxiliares).
- También se incorporan datos desde el sistema *ERP*:
 - Turnos de las cuadrillas.
 - Disponibilidad de equipos auxiliares.

Ejemplo de Tarea y sus atributos

Atributo	Descripción
ID Orden de Trabajo	4000001
ID Tarea	001
Descripción OT	Cambio de suspensión trasera lado izquierdo
Descripción Tarea	Retiro de suspensión trasera
Fecha Inicio Extrema	2024-11-29
Fecha Requerida	2024-12-30
Cuadrilla	BM001
Equipo Auxiliar	Alza Hombres Telescópico
Duración (horas)	6
Criticidad	2
Cantidad de Trabajadores	4

Cuadro: Ejemplo de Tarea y sus atributos

Consideraciones al programar

- **Restricciones de precedencia** dentro de la orden de trabajo (OT).
- **Cuadrillas:**
 - Tienen esquemas de turnos (disponibilidad limitada).
 - Cuentan con una capacidad (número de trabajadores) que determina cuántas tareas en paralelo pueden atender.
- **Equipos auxiliares** son recursos unitarios (no se comparten en simultáneo).
- **Cronograma base:** Se parte de cronogramas anteriores para ajustarlos e incorporar nuevas tareas.

Requisitos de la solución

- **Eficiencia:** Procesamiento rápido y generación de cronogramas en tiempo real.
- **Escalabilidad:** Capacidad de manejar grandes volúmenes de tareas y recursos.
- **Flexibilidad:** Adaptación a cambios de último minuto y restricciones dinámicas. La solución debe ser capaz de partir de un cronograma base y ajustarlo, minimizando la cantidad de cambios.
- **Usabilidad:** Interfaz amigable y fácil de usar para el personal de programación.

6. Diseño de la solución

Flujo de Trabajo de la Solución

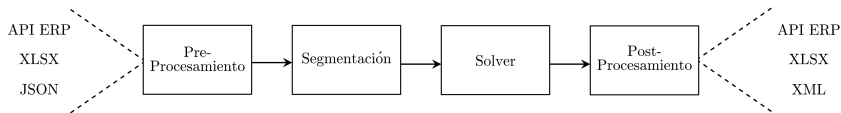


Figura: Flujo de Trabajo de la Solución.

① Pre-Procesamiento

② Segmentación en Subconjuntos

③ *Solver*

④ Post-Procesamiento

1. Pre-Procesamiento

- **Objetivo:** Limpiar y preparar la información procedente de las fuentes de datos (*ERP, Microsoft Project*, u otros formatos).
- Operaciones clave:
 - **Corrección de discrepancias:** Ajuste automático de errores (ej. fechas incoherentes).
 - **Cuantización de ventanas horarias:** Expresar duraciones en intervalos de 15 minutos para optimizar el modelado.
 - **Estandarización en JSON:** Consolidación de datos (duraciones, recursos, dependencias) en un formato coherente.

2. Segmentación en Subconjuntos

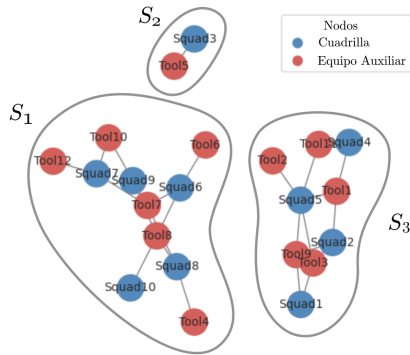


Figura: Ejemplo de segmentación a través de componentes conexos.

Metodología de Segmentación

- **Objetivo:** Reducir la complejidad del problema al dividirlo en grupos más pequeños.
- **Método:**
 - ① **Construcción del grafo:** Cuadrillas y equipos como nodos; existe arista si hay tareas que usan ambos recursos de forma simultánea.
 - ② **Componentes conexos:** Identificación de grupos interrelacionados que se tratarán como un subconjunto.
 - ③ **Formación de subconjuntos:** Cada componente conexo se resuelve de manera independiente en el *solver*.

3. *Solver*: Construcción de *input*

- Uso de *OR-Tools* (CP-SAT solver) para modelar y resolver el *RCPSP*.
- **add_hint**: Permite incorporar soluciones anteriores o ajustes del usuario como punto de partida.
- Cada subconjunto se resuelve de forma independiente, pudiendo ejecutarse en paralelo (*joblib*, hilos, múltiples núcleos).
- **Ventajas de la paralelización:**
 - Aprovecha mejor los recursos de hardware.
 - Reduce el tiempo total de ejecución.
- Se obtiene una solución parcial por cada subconjunto, que luego se unifica.

4. Post-Procesamiento

- **Objetivo:** Consolidar y preparar la salida del *solver* para su integración y análisis final.
- Salidas principales:
 - **Archivos XML para MS Project:** Permite incorporar el cronograma a entornos de gestión existentes.
 - **Archivo XLSX (Gantt):** Visualización rápida de la programación en hojas de cálculo.
 - **Integraciones con APIs:** Conexión automática con sistemas *ERP* o herramientas empresariales.

7. Resultados y análisis

Conjunto de datos: Características generales

- Datos reales provistos por la empresa colaboradora.
- Período de planificación: Del 17 de julio al 3 de agosto de 2023.

Atributo	Valor
Número total de tareas	4.898
Número de órdenes de trabajo (OT)	1.055
Fecha mínima	2023/07/17
Fecha máxima	2023/08/03
Duración promedio (horas)	1,92
Número de cuadrillas	31
Número de equipos auxiliares	16

Cuadro: Resumen de las características del conjunto de datos.

Resultados para distintos α y β

- La función objetivo combina:

$$\text{mín } Z = \alpha \cdot \text{makespan} + \beta \cdot \sum_{i=0}^n w_i \cdot (1 - x_i)$$

- α : Peso del *makespan*.
- β : Penalización por no programar tareas (enfocada en criticidad).
- Múltiples experimentos con distintas combinaciones (α, β) para analizar:
 - Número total de tareas programadas.
 - Tareas no programadas por criticidad.
 - *Makespan*.
 - Tiempo de solución.

Resultados Generales (α - β)

Conf.	α	β	Tareas Prog.	Tareas no Prog.	Tiempo Sol. (s)	Makespan
(a)	1	0	0	4.898	3,43	0
(b)	0	1	4.854	44	4,35	1.072
(c)	1	1	4.658	240	8,42	492,75
(d)	1	2	4.831	67	9,95	516,25

Cuadro: Resultados con diferentes configuraciones de α y β .

Distribución de Tareas no Programadas por Criticidad

Conf.	α	β	Criticidad 1	Criticidad 2	Criticidad 3
(a)	1	0	666	2.403	1.829
(b)	0	1	0	8	36
(c)	1	1	21	59	160
(d)	1	2	0	12	55

Cuadro: Tareas no programadas, según criticidad, en cada configuración.

Elección de α y β

Conclusiones:

- (a): Minimiza *makespan*, pero no programa ninguna tarea.
- (b): Maximiza tareas programadas (incluye todas las críticas), pero *makespan* muy alto.
- (c): Menor *makespan* pero muchas tareas críticas no programadas.
- (d): Equilibrio entre ambas metas: 4.831 tareas programadas (0 tareas críticas no programadas) y 516,25 h de *makespan*.

Optimización mediante Segmentación en Subconjuntos

- El objetivo es reducir la complejidad del problema y **acelerar el tiempo de solución**.
- A modo de *benchmark*, el tiempo de solución sin segmentación fue de **9,95 s**.
- Se dividió el total de 4.898 tareas en **14 subconjuntos**.
- Reducción de complejidad y paralelización del *solver*.

Segmentación en Subconjuntos

Subc.	Tareas	% del total	Tiempo Sol. (s)
S1	2.598	53 %	4,34
S2	674	14 %	0,65
S3	319	7 %	0,43
S4	254	5 %	0,12
S5	198	4 %	0,01
S6	136	3 %	0,01
S7	122	2 %	0,00
S8	110	2 %	0,01
S9	98	2 %	0,00
S10	87	2 %	0,00
S11	84	2 %	0,01
S12	78	2 %	0,00
S13	72	1 %	0,00
S14	68	1 %	0,00

Cuadro: Resultados de la ⁵⁰segmentación en subconjuntos.

Tiempo total y conclusiones de la segmentación

- El tiempo de solución total equivale al máximo de todos los subconjuntos:
 $\sim 4,34$ s.
- Paralelización:
 - Cada subconjunto se procesa en hilos o núcleos distintos.
 - Aprovechamiento óptimo del hardware disponible.
- Subconjuntos más pequeños ($< 5\%$) se resuelven casi instantáneamente.
- **Conclusión:** El enfoque modular y paralelo disminuye significativamente el tiempo total de ejecución sin sacrificar la calidad del cronograma.

8. Conclusiones y pasos a seguir

Conclusiones Generales

- Se alcanzaron los objetivos propuestos, logrando una **solución eficiente** para la programación automatizada de mantenimiento.
- En un conjunto de 4.898 tareas:
 - 98,6 % de tareas programadas (solo 67 no programadas).
 - **Todas** las tareas de criticidad 1 fueron incluidas.
 - *Makespan*: 21,5 días.
- **Tiempo total de cálculo:** 6,12 segundos (incluye pre-procesamiento, segmentación, resolución y post-procesamiento).

Cronograma Resultante en MS Project

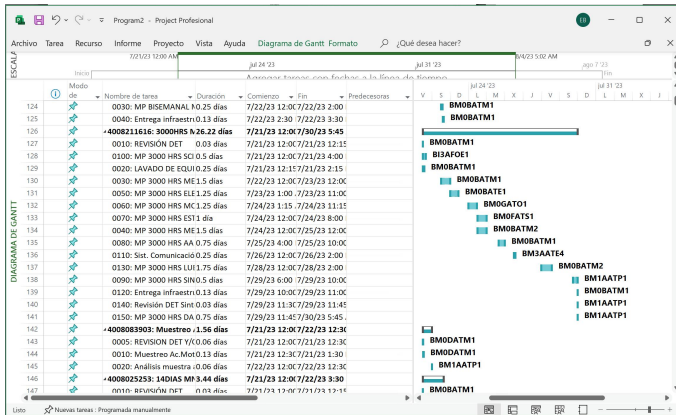


Figura: Captura de Cronograma Resultante en MS Project

Impacto estimado sobre el proceso de programación

- La herramienta automatiza completamente la generación del **borrador inicial**, eliminando la necesidad de hacerlo manualmente.
- Permite realizar **ajustes finos** sobre el borrador inicial, como:
 - Fijar tareas en horarios específicos.
 - Modificar disponibilidad de recursos.
 - Incorporar tareas de último minuto.

Durante estos ajustes, el programa adapta automáticamente las tareas relacionadas para garantizar un cronograma viable, reduciendo a la mitad el tiempo necesario para este proceso.

- Se estima por lo tanto una reducción total del $\sim 57,6\%$ en el tiempo necesario para la creación del cronograma final.

Limitaciones y consideraciones

- **Disponibilidad de materiales y repuestos:**
 - Principal error detectado.
 - No sistematizado en el *ERP*, por lo que no fue incluido en la solución.
- Para integrar este aspecto se requiere:
 - Paso previo de **gobernanza de datos**.
 - Procesos robustos de manejo en bodega.
 - Sistematización de información de repuestos.
- Sin estos pasos, la herramienta no reduce errores por falta de materiales, requiriendo colaboración y compromiso de otras áreas.

Pasos a seguir: Módulos de Desarrollo de la Solución

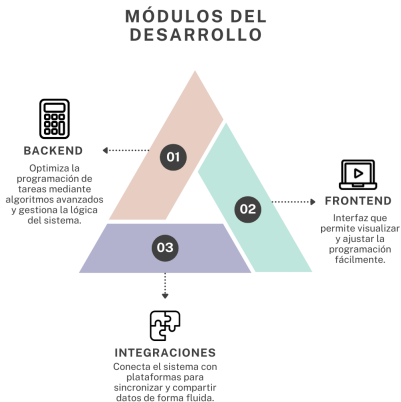


Figura: Propuesta de Módulos de Desarrollo de Solución.

Estructura de la Solución

- **Backend:**

- Coordina y paraleliza la ejecución del *solver*.
- Centraliza los datos y gestiona la comunicación con la base de datos.
- Garantiza la escalabilidad y estabilidad del sistema.

- **Frontend:**

- Ofrece una interfaz clara e intuitiva para los programadores.
- Facilita la carga de datos, la edición de cronogramas y la revisión de resultados.
- Permite visualizar la información de forma amigable y comprensible.

- **Integración con la plataforma de la empresa:**

- Conecta la solución con sistemas existentes (p.ej. ERP).
- Asegura la importación y exportación de datos de manera automatizada.
- Mantiene la información sincronizada y actualizada en todo momento.

Reflexiones finales

- **Objetivo cumplido:** Se desarrolló una herramienta capaz de automatizar la programación de actividades de mantenimiento con alta eficiencia.
- **Reducción de errores y carga de trabajo:** Se estima que la herramienta permitirá reducir significativamente el tiempo necesario para las tareas de programación además de disminuir errores.
- **Limitaciones:** La falta de sistematización de información sobre materiales y repuestos es una limitación importante que debe ser abordada en futuras etapas.
- **Adopción en la realidad operativa:** Para que la herramienta sea incorporada de manera efectiva, se requiere:
 - Una **interfaz intuitiva** y fácil de usar.
 - **Integración** confiable con las plataformas de la empresa.

Bibliography